AthenaeumAI

An AI-Powered Adaptive Learning Platform

Technical Report — Version 1.0

Pratham Kashyap

July 15, 2026

A comprehensive technical report covering system architecture, adaptive learning algorithms, AI integration, database schema, background processing, security model, testing strategy, and deployment infrastructure.

Abstract

AthenaeumAI is a full-stack, AI-powered learning platform that generates quizzes, flashcards, analytics, and tutoring experiences from uploaded study material. Unlike static quiz generators, AthenaeumAI tracks per-topic mastery using exponential decay models, implements the SM-2 spaced repetition algorithm for flashcard scheduling, and employs a Retrieval-Augmented Generation (RAG) pipeline for hallucination-resistant AI tutoring. The system uses a containerized microservices architecture with React, Node.js/Express, MongoDB (with replica-set transactions), Redis, and BullMQ for durable asynchronous processing. This report documents the complete technical design, implementation decisions, and testing methodology of the v1.0 release.

Contents

1 Introduction

The proliferation of digital learning tools has not kept pace with the pedagogical insight that *how* knowledge is reviewed matters as much as *what* is studied. Most quiz platforms generate questions statelessly — they produce output and immediately forget the learner. AthenaeumAI was developed to address this gap by building a platform that continuously adapts to individual learning trajectories.

1.1 Problem Statement

Traditional learning management systems suffer from three fundamental limitations:

1. **Static question banks** — Questions are manually authored and do not scale with new material.
2. **No retention modeling** — Systems cannot predict which topics a learner is about to forget.
3. **Disconnected feedback** — Quiz results, flashcards, and tutoring exist as isolated features rather than a unified adaptive pipeline.

1.2 Solution Overview

AthenaeumAI solves these problems by integrating:

- Automated quiz generation from arbitrary PDF content via LLMs.
- Per-topic mastery tracking with exponential decay-based retention scoring.
- SM-2 spaced repetition for flashcard scheduling.
- A RAG-grounded AI tutor that answers strictly from uploaded material.
- Asynchronous background processing for material indexing and mistake analysis.

2 System Architecture

The platform follows a containerized, service-oriented architecture orchestrated via Docker Compose. Five containers collaborate at runtime:

1. **Frontend** — React 18 SPA served via Nginx (port 8080).

2. **Backend** — Express 5 REST API (port 5000).
3. **Worker** — Dedicated BullMQ consumer process.
4. **MongoDB** — Single-node replica set for transaction support.
5. **Redis** — Message broker for BullMQ job queues.

2.1 Request Flow

A typical user interaction follows this path:

1. The React frontend issues an authenticated REST request.
2. The Express API validates the JWT, processes the request, and performs synchronous database operations within a Mongoose transaction.
3. For asynchronous work (material indexing, attempt synchronization), the API enqueues a BullMQ job to Redis.
4. The dedicated Worker process dequeues and processes the job, updating MongoDB accordingly.
5. The frontend polls or re-fetches data via TanStack Query to reflect the updated state.

2.2 Technology Decisions

Layer	Technology	Rationale
Frontend	React 18 + Vite	Fast HMR, lazy-loaded routes, TypeScript type safety
UI Components	shadcn/ui + Radix	Accessible, composable primitives
Styling	TailwindCSS	Utility-first CSS with design token consistency
Data Fetching	TanStack Query	Automatic caching, background refetching, optimistic updates
Backend	Express 5	Mature HTTP framework with async error handling
Database	MongoDB + Mongoose 9	Flexible document model with replica-set transactions
Queue	BullMQ + Redis	Durable job processing with retries, backoff, and deduplication

AI	Groq API (Llama 3.3 70B)	Low-latency inference with structured JSON output
Validation	Zod	Runtime schema validation for API inputs and environment
Logging	Winston	Structured JSON logging with log levels
Auth	JWT (HttpOnly cookies)	Stateless authentication with refresh token rotation

3 Database Schema

The application persists data across 12 Mongoose models:

Model	**Purpose**
User	Authentication credentials, profile metadata
RefreshToken	JWT refresh token storage for rotation
StudyMaterial	Uploaded PDF metadata and extracted text
MaterialChunk	Indexed text segments for RAG retrieval
Quiz	Generated questions with difficulty metadata
QuizAttempt	User answer submissions with scoring
UserProgress	Per-topic mastery, confidence, weakness scores
LearningEvent	Timestamped learning activity log
FlashcardSet	SM-2 flashcards with scheduling state
ReviewQueue	Prioritized review items per user
Notification	System notification payloads

3.1 Key Schema Relationships

- `UserProgress` aggregates mastery per topic per user, updated atomically after each quiz attempt.
- `MaterialChunk` documents are created by the background worker during material indexing and referenced by the RAG tutor pipeline.
- `FlashcardSet` contains an array of individual cards, each carrying SM-2 scheduling state (ease factor, interval, repetitions, next review date).

4 Adaptive Learning Engine

4.1 Per-Topic Mastery Tracking

After each quiz attempt, the `progressService` recalculates per-topic metrics:

- **Mastery** — Weighted rolling average of accuracy across attempts on a given topic.
- **Confidence** — Statistical confidence based on attempt count and consistency.
- **Weakness Score** — Inverse function of mastery penalized by recency of mistakes.

4.2 Retention Decay Model

The analytics engine computes a *retention score* per topic using exponential decay:

$$R(t) = C \cdot e^{-\lambda t}$$

where C is the topic confidence, t is the number of days since the last practice session, and $\lambda = 1/30$ is the decay constant (calibrated for a 30-day half-life). Topics with low retention scores are automatically surfaced in the review queue.

4.3 Estimated Readiness

The dashboard displays an overall *readiness* metric computed as:

$$Readiness = 0.55 \cdot Mastery + 0.25 \cdot Confidence + 0.20 \cdot Retention$$

The mastery weight (0.55) is intentionally dominant because demonstrated knowledge is the strongest predictor of assessment performance.

4.4 Weak Topic Detection

Topics are flagged as "weak" if:

- Weakness score exceeds 35, **or**
- Mastery is below 65% (even if weakness score is low).
- Topics with zero attempts are excluded to avoid false positives.

Results are sorted by weakness score in descending order and capped at 6 topics.

5 Spaced Repetition — SM-2 Algorithm

The flashcard system implements the SuperMemo SM-2 algorithm with the following parameters:

Rating	Quality	Effect
Again	1	Reset repetitions to 0, interval to 0 (same-day review), decrease ease factor
Hard	2	Increment repetitions, interval 1 day on first rep, slightly lower ease
Good	4	Standard progression, ease factor multiplier applied after second repetition
Easy	5	1.3x bonus multiplier on interval, increase ease factor

5.1 Ease Factor Constraints

- Minimum (floor): 1.3
- Maximum (ceiling): 3.0
- Default starting value: 2.5

The interval is always constrained to be ≥ 0, and unknown ratings fall back to "good" (quality 4) to prevent scheduling failures.

6 AI Integration

6.1 Quiz Generation Pipeline

The quiz generation service (`aiQuizService.js`) orchestrates:

1. Text chunking to respect Llama 3.3's context window.
2. Structured prompt construction requesting JSON-formatted multiple-choice questions.
3. Response parsing with quality filtering (`qualityFilter.js`).
4. Persistence within a Mongoose transaction.

6.2 Quality Filtering

Each generated question receives a score from 0–10 based on:

- Structural validity (question string, 4 options, valid answer index).
- Minimum question length (≥ 40 characters).
- Absence of ambiguous options ("all of the above", "none of the above").
- No duplicate options.
- Presence and quality of explanation (≥ 30 characters).

Questions scoring below 5 are discarded before persistence.

6.3 RAG-Grounded AI Tutor

The AI tutor prevents hallucination through strict retrieval grounding:

1. The user's query is combined with session history for context continuity.
2. The `embeddingService` retrieves relevant `MaterialChunk` documents from the user's uploaded material.
3. A constrained system prompt instructs the LLM to answer *only* from the provided chunks, refusing to speculate beyond the source material.

7 Background Processing — BullMQ

7.1 Architecture

The BullMQ pipeline consists of three components:

- **Producer** (`jobQueue.js`) — Enqueues jobs with exponential backoff, bounded retention, and typed error handling.
- **Consumer** (`worker.js`) — Dedicated Node.js process with concurrency 5, graceful shutdown (SIGINT/SIGTERM), and strict payload validation.
- **Broker** (Redis) — Durable message persistence between producer and consumer.

7.2 Job Types

Job Type	Description
`INDEX_MATERIAL`	Parses uploaded PDF text into `MaterialChunk` documents for RAG retrieval
`SYNC_ATTEMPT`	Updates `UserProgress`, records `LearningEvents`, performs mistake analysis, and populates the review queue
`REBUILD_REVIEW_QUEUE`	Recalculates and re-ranks the user's review queue based on current mastery and flashcard due dates

7.3 Reliability Hardening (v1.0)

The v1.0 release includes several reliability improvements:

- **Explicit enqueue errors** — `QueueEnqueueError` (503) is thrown and propagated to the HTTP response if Redis cannot accept a critical job.
- **Awaited enqueue** — Critical job submissions (`INDEX_MATERIAL`, `SYNC_ATTEMPT`) are `awaited` by their controllers.
- **Strict worker validation** — Unknown job types and missing Mongo dependencies now throw errors (triggering BullMQ retries) instead of silently succeeding.
- **Bounded retention** — Completed jobs are retained for 500 entries or 7 days; failed jobs for 500 entries or 30 days.
- **Deduplication** — `REBUILD_REVIEW_QUEUE` jobs use per-user deduplication IDs to prevent unbounded job creation from dashboard reads.

8 Authentication and Security

8.1 Authentication Flow

1. User registers with name, email, and password (validated via Zod schemas).
2. Password is hashed before persistence.
3. On login, a JWT access token and refresh token are issued.
4. Access tokens are sent as HttpOnly cookies to prevent XSS access.
5. Refresh tokens support rotation for session continuity.

8.2 Security Middleware

- **Helmet** — Sets security-related HTTP headers.
- **CORS** — Restricted to the configured `FRONTEND_URL`.
- **Rate Limiting** — Applied to AI-intensive endpoints to prevent abuse.
- **Multer** — File upload validation and size constraints.
- **Request Validation** — Zod-based middleware validates all API inputs at the route level.

9 Testing Strategy

9.1 Backend Unit Tests

The backend includes 118 unit tests across 6 test suites:

- `analyticsService.test.js` — Retention decay, readiness formula, weak topic detection, accuracy trend bucketing.
- `flashcardService.test.js` — SM-2 algorithm correctness for all rating levels, edge cases, and clamp boundaries.
- `qualityFilter.test.js` — Scoring logic, structural disqualifiers, penalty stacking.
- `reviewQueueService.test.js` — Item classification, pagination, snooze behavior.
- `topicNormalizationService.test.js` — Abbreviation mapping, title case fallback, null handling.
- `jobQueue.test.js` — Enqueue success, typed error propagation, function payload rejection.

9.2 Integration Tests

- `api.test.js` — 21 tests covering health checks, signup, login, protected route rejection, analytics, flashcards, and review queue endpoints.
- `bullmq.test.js` — Queue lifecycle, enqueue failure, retry behavior, bounded retention, and deduplication (requires live Redis).

9.3 Frontend Tests

- Vitest unit tests for component logic.
- Playwright E2E tests for critical user flows.

9.4 CI Pipeline

GitHub Actions runs on every push and pull request:

1. Install dependencies (frontend and backend).
2. Run ESLint.
3. Build the frontend (Vite production build).
4. Syntax-check the backend (`node -check`).
5. Execute all backend unit and integration tests (with Redis service container).

10 Deployment

10.1 Docker Compose

The `docker-compose.yml` orchestrates five services:

- **mongo** — MongoDB with single-node replica set initialization.
- **redis** — Redis 7 Alpine.
- **backend** — Express API server.
- **worker** — BullMQ consumer (same image, different entrypoint).
- **frontend** — Nginx serving the Vite production build.

10.2 Production Recommendations

Service	Recommendation
MongoDB	Atlas M10+ with replica set
Redis	Upstash (serverless) or AWS ElastiCache
Backend + Worker	Railway, Render, or AWS ECS

Frontend	Vercel or AWS Amplify

11 Future Work

11.1 Phase 6: Semantic RAG

The current RAG pipeline uses hash-based text retrieval. Phase 6 will introduce:

- Dense vector embeddings via Sentence Transformers.
- Qdrant vector database for similarity search.
- Hybrid retrieval combining keyword and semantic search.
- Lightweight cross-encoder reranking for precision.

11.2 Phase 7: Advanced Features

- Multi-modal document support (images, diagrams).
- Collaborative study sessions.
- Transactional outbox pattern for guaranteed async consistency.

12 Conclusion

AthenaeumAI demonstrates that a student portfolio project can reach production-grade architectural quality. The platform integrates adaptive learning algorithms, durable background processing, RAG-grounded AI tutoring, and comprehensive testing into a cohesive full-stack application. The v1.0 release represents a stable, deployable product with clear extension points for semantic search and advanced pedagogical features.

End of Document

AthenaeumAI Technical Report v1.0

Pratham Kashyap — July 15, 2026